



ChatGPT Conversation Export Extension – UX/UI Specification

Introduction & Problem Statement

Exporting ChatGPT conversations should be simple and powerful, yet current solutions are either too basic or too granular. The existing MVP extension injects a small **Export** button into ChatGPT's interface and can output Markdown, HTML, or JSON. However, its developer-oriented UI (with a checkbox for every content block) overwhelms casual users and lacks a friendly "chat-style" preview. Users need an easy way to **archive entire chats**, **capture highlights or code snippets**, and **select conversation ranges** – all with a polished, intuitive interface. This specification defines a comprehensive UX/UI design for the extension's next version, balancing **one-click simplicity** for normal users with **granular control** for power users.

Key design goals:

- **Seamless Integration:** The export tools should feel like a natural part of ChatGPT's UI, minimally invasive until needed.
- **One-Click Simplicity:** Common tasks (save whole chat, single answer, code only, etc.) should require minimal effort (preferably one click).
- **Power User Features:** Advanced selection and filtering (by message, by content type, by code version) should be available without cluttering the simple experience.
- **Familiar Chat Aesthetics:** Exports (and previews) should use a left-right chat bubble layout (user on right, assistant on left) for readability and familiarity ¹.
- **Resilience & Light Footprint:** The extension should not rely on brittle page selectors (to survive ChatGPT UI updates) and should only activate when used, to avoid performance hits.
- **Accessibility & Theming:** The UI must support dark/light mode, high contrast, keyboard navigation, and screen readers, ensuring inclusive use.

Research Insights: Existing Tools & Patterns

Extensive research was conducted on existing ChatGPT export tools and related UI patterns to inform this design:

- **ShareGPT:** A popular extension focused on sharing. It adds a small "Share" button within ChatGPT that, with one click, opens a new tab containing the full conversation on a shareable URL ². *Strengths:* Extremely simple – one click yields a shareable chat transcript. *Weaknesses:* Limited to sharing via link; no options for local save or partial selection.
- **ChatGPT to Notion Exporter:** This extension (e.g. by gptsave) allows saving chats directly to Notion. Triggered from the browser's extension menu, it opens a dialog with multiple export options (copy to clipboard, save as image, or export to Notion) ³. It formats the conversation neatly before sending to Notion and supports alternate outputs like an image screenshot. *Strengths:* Preserves formatting

and integrates with a note-taking tool; offers multiple output types. *Weaknesses:* Requires linking a Notion account (extra step); UI is a popup separate from the ChatGPT page, which disrupts flow.

- **“ChatGPT Export & Share” Extension:** A minimal tool available outside official stores that injects quick action buttons (e.g. *Generate PNG, Download PDF, Share Link*) directly into the ChatGPT interface. This lets users instantly save a PNG screenshot, PDF, or get a share link. *Strengths:* One-click outputs for common formats, very easy to use ⁴. *Weaknesses:* Lacks customization or partial export (all or nothing); PDF/image outputs sometimes had formatting issues per user reports.
- **ChatGPT Prompt Genius:** A browser extension with a broader feature set; for exports it adds a **“Export & Share”** menu item in ChatGPT’s left sidebar. Clicking it reveals options to download the conversation as a PDF, PNG image, Markdown file, or to create a shareable link ⁵. This UI integrates into ChatGPT’s own menu (feeling native) and includes Markdown which users appreciated for easy reuse in web content ⁶. *Strengths:* Multi-format support within a clean dropdown; feels built-in to ChatGPT UI. *Weaknesses:* The sidebar menu might be less discoverable than an always-visible button; no fine-grained selection of content (exports the whole chat by default).
- **Superpower ChatGPT:** A power-user extension that among many features adds an **Export** button for conversations ⁷. It allows saving chats in formats like TXT, Markdown, JSON, and PDF ⁷. It even supports selecting multiple conversations at once to export in bulk. *Strengths:* Covers all common formats (including PDF which users often request ⁸) and even multi-chat export. *Weaknesses:* Geared towards heavy users – requires using its custom interface (folders, multi-select); not focused on in-conversation partial exports.
- **ExportGPT (ChatOpenAI):** A recent all-in-one export tool with a sidebar UI. It supports Markdown, PDF, HTML, Excel, image, etc., and provides a preview/edit interface before saving ⁹. Notably, it offers flexible selection: users can “select all AI answers”, invert selections, or export only the assistant’s responses ¹⁰. *Strengths:* Very feature-rich (multiple formats, selection toggles) and an in-app preview editor ⁹. *Weaknesses:* Some features are paywalled and certain exports (PDF, images) reported reliability issues (missing images, layout problems) ¹¹ ¹². The plethora of options could overwhelm users looking for a quick save.

Other patterns considered:

- **Messaging App Exports:** WhatsApp offers “Export Chat” as plain text (usually via email) – functional but losing rich media/formatting. Slack allows admins to export data in JSON or TXT ¹³, but average users cannot easily save a nicely formatted chat transcript via the UI. These examples show that *raw exports exist, but user-friendly formatted exports are not standard in chat apps*. Our tool can fill that gap for ChatGPT, providing clean, shareable transcripts.
- **Note-Taking & Web Clippers:** Tools like Evernote or Notion Web Clipper allow users to save selected content from a page. They emphasize **highlighting important parts** and preserving content structure. This informs our “highlights” feature – users should be able to pick out key answers or lists and save them without manual copy-paste.
- **Code Snippet Managers:** Developers often use IDEs or tools like Gists to manage code versions. The concept of grouping code by file and showing differences between versions is familiar (akin to a Git diff or code review interface). No current ChatGPT exporter auto-diffs code changes, but adding this would cater to

developer workflows. We take inspiration from version control UIs to design how code blocks could be compared (e.g., highlighting added/removed lines between successive versions of a snippet).

User Expectations: Through forums and feedback, we validated that **casual users** mostly want a one-button solution to save or share an entire conversation (for memory or sharing) ² ⁸. They value *simplicity* and *fidelity* (the saved output should look like the chat, with minimal hassle). In contrast, **developers and researchers** desire more control: the ability to extract just the assistant's answers or just code, to compile data for further use, or to archive specific insights. They are willing to use more complex UIs if it yields cleaner or more customized results. The challenge (and opportunity) is to **serve both groups in one extension** by separating "simple" and "advanced" modes.

Key Pain Points in Current MVP:

- **Overwhelming Granularity:** Requiring the user to check dozens of boxes (one per paragraph, list, code block, etc.) to select content is tedious. In long chats, this is impractical (selecting 100+ blocks) and deviates from how users think of conversations (they think in messages or turns, not individual HTML elements).
- **Lack of Visual Preview:** Users have no easy way to see what the exported result will look like in the current UI – the output is either downloaded blindly or shown as raw Markdown/HTML. This can be confusing for non-technical users who just want a nicely formatted transcript.
- **No Quick Defaults:** The current design doesn't offer one-click "Export All". Every export requires fiddling with checkboxes, whereas many users simply want the whole conversation or a single answer without extra steps.
- **Limited Contextual Actions:** There's no way to export a specific answer on the fly (e.g. via right-click or a small button on that answer). Users have to open the panel and then find that answer's blocks to select. This is friction for the "quick snippet" scenario.
- **Styling and Theming Issues:** The existing panel is very bare-bones (checkboxes and plain text) – it doesn't align with ChatGPT's dark theme or UX. This visual dissonance can erode trust. Also, accessibility (keyboard navigation, proper labels) wasn't a focus.

These findings guided the new design, combining the best ideas from existing solutions (one-click actions, integrated previews, multi-format support) while addressing their shortcomings (too basic vs. too complex, lack of selection finesse, etc.).

Overall Flow & UI Structure

The extension's interaction flow is designed to be **discoverable yet unobtrusive**:

- **Export Button (Trigger):** A small floating button labeled "Export Chat" will be fixed at the bottom-right corner of the ChatGPT interface (above the prompt input area). It uses a subtle icon (e.g. a download tray or share symbol) and matches the existing UI color scheme (semi-transparent dark background in Dark mode). This button is only displayed when a conversation is present (to avoid clutter on empty/new chats). It's analogous to how some chat tools overlay an action button for extras, ensuring familiarity. *(This approach is inspired by ExportGPT's "convenient sidebar button" concept ⁹, but we place it bottom-right for visibility without interfering with the left sidebar or chat flow.)* Clicking this button launches the main UI panel.
- **Main UI Panel:** Upon clicking **Export Chat**, an overlay panel opens *within* the ChatGPT page. This panel slides up or fades in, covering a portion of the screen (centered or aligned to right side). It

behaves like a modal: the rest of the page is dimmed but visible behind, so users see the context of their current chat. The panel can be moved or resized (for advanced users who may want to see the chat behind it), but by default it occupies about 60% of the screen (enough to show a preview of the conversation). There will be a close “X” button to dismiss it without exporting (or pressing Escape). Using an in-page overlay ensures the user stays in context and can easily refer back to the chat if needed, without juggling browser tabs (a conscious choice since opening in a new tab, while offering more space, was deemed less seamless for quick exports). For lengthy editing or if the user prefers, an “Open in New Tab” option will be provided to pop the preview into a separate window – but the primary flow is inline.

- **Mode Toggle – Simple vs. Advanced:** The panel’s content is divided into two modes, accessible via tabs or toggle buttons at the top: **Simple** and **Advanced**. The default is **Simple** for all users. This tabbed design ensures casual users aren’t confronted with complex options upfront, while power users can switch to Advanced easily. The toggle is clearly labeled (e.g. “ Simple Export” and “⌕ Advanced Export”). It remembers the last-used mode (so a developer can always open directly in Advanced if they used it last, whereas a regular user will stick to Simple).
- **Context Menu Integration:** In addition to the main button, the extension registers a context menu item within the browser for the ChatGPT site. When users right-click on an assistant answer (message bubble), they will see an option: **“Export this answer”**. Selecting this bypasses the main UI for speed – it will immediately harvest that single answer (with minimal surrounding context) and trigger a download in the default format (e.g. Markdown). This fulfills the “quick snippet” scenario. For example, if a user just received a useful code answer, a right-click allows saving that answer as a Markdown file instantly (or copying it to clipboard, as an alternative action). This contextual shortcut aligns with typical user behavior of selecting or right-clicking content to save it. *(We limit this menu to assistant answers, not user prompts, under the assumption that the user usually wants to save AI outputs. If needed, a similar “Export this prompt+answer pair” could be offered.)*
- **Minimal Impact on ChatGPT UI:** Other than the floating Export button and context menu, we avoid altering ChatGPT’s existing interface. Unlike Prompt Genius which added a permanent sidebar item ⁵, our approach keeps the UI clean until activation. This reduces the risk of UI conflicts and “UI fatigue” (users not wanting their chat screen cluttered with too many extension icons). The floating button approach is also used by some screenshot extensions and feels familiar ¹⁴. It will have a tooltip (“Export or save this conversation”) for clarity.

Simple Mode Design

The **Simple Mode** is geared toward everyday users who want quick, sensible exports with minimal configuration. It provides a **chat-style preview** and preset export options:

Chat-Style Preview Panel

Upon opening the panel in Simple mode, the user is presented with a live preview of the conversation in a chat format:

- **Bubble Layout:** Messages are rendered as speech bubbles or blocks – **assistant messages on the left, user messages on the right**, mirroring common chat applications ¹. This layout instantly conveys the dialogue structure. Each message is separated with a small margin, and optionally, a tiny avatar or role label (e.g. “You” vs “ChatGPT”) can be shown to further distinguish authors ¹⁵. We will use subtle color differences: for example, user bubbles in a blue/gray shade (right side), assistant bubbles in a green/gray shade (left side), similar to iMessage or WhatsApp styling. This visual division helps readability and aligns with user expectations for a “chat transcript.”
- **Preserved Formatting:** Within each message bubble, content is displayed with proper formatting – code blocks in a monospace fenced block with a light background, lists as bullet points, tables with borders, math formulas rendered, and images (if any) shown as thumbnails. Essentially, we use the same structured JSON (from the DOM) to reconstruct the conversation in this viewer. This preview is not an image; it’s live HTML/CSS so that text can be selected and links (if any) clicked. The styling will mimic ChatGPT’s own, but we have control to ensure the entire conversation fits into a printable view if needed (e.g. using a white background for exported PDF/HTML if chosen).
- **Scrolling & Range Selection:** If the conversation is long, the preview is scrollable. The panel header remains fixed so the export controls are always visible. To support **partial conversation exports (replay/cut ranges)** without dozens of checkboxes, Simple mode implements a *range selection* mechanic:
 - There is a small “**Select Range**” toggle button in the preview header. When activated, the preview enters a selection mode where hovering over a message highlights it, and the user can click two messages to mark the **start** and **end** of a range. For instance, click the message at which to start export, then click the message at which to end – those two messages get a highlight (e.g. a blue outline labeled “Start” and “End”). All messages outside this range become semi-transparent or are hidden, so the user sees exactly the segment to export. The user can adjust by moving the start/end markers (clicking elsewhere or dragging them).
 - Another method for range selection: the user can hold Shift and click on a second message, which will select all messages in between (similar to selecting multiple items in a list). A subtle indication (like a shaded background for selected messages) will appear.
 - When satisfied, the user clicks “Done” (or toggles off Select Range), and the preview locks to showing only that subset. An info line might indicate “Exporting messages 3–7 of 12”. This approach allows selecting “from here to here” in a natural, visual way – addressing the replay scenario. It eliminates having to tick 10 boxes in Advanced mode; instead the user directly *sees* the portion they will export. If the user wants the entire conversation (the most common case), they simply don’t use Select Range (by default all messages are included).
- **Content Filters (Highlights):** Simple mode also offers quick filters to capture **specific content types** (fulfilling the “highlights” and “structured outputs” use cases). At the top of the preview, there is an “**Export Only...**” dropdown or set of toggle buttons:

- Options include **“Assistant Answers Only”**, **“Code Snippets”**, **“Tables”**, **“Bullet Lists”**, and **“Images”**. If the user selects “Assistant Answers Only,” the preview will hide all user prompt bubbles – showing a continuous flow of assistant responses (useful for reading just the AI’s narrative or for sharing just the AI content). Selecting “Code Snippets” will hide all non-code content, showing only the code blocks from each assistant message (with perhaps a small note of which message it came from). Similarly, “Tables” will show only the tables, etc. Multiple filters can be combined if needed (the UI will ensure at least one is always active; if none toggled, it’s equivalent to no filter). These filters dynamically update the preview so the user can verify the output.
- Example: If a user wants to **capture only the key points** from a chat, and the assistant’s answers contain bullet lists of summary points, they could toggle “Bullet Lists” – then the preview might display only those bulleted sections from across the conversation, concatenated. The context of which answer they came from might be indicated by a small heading or a divider. This feature is inspired by web highlighters and clippers that let users extract structured pieces of content, and by user requests to easily get *just* the useful parts (e.g. just the final answer, or just the summary table).
- A **“Highlights”** option could also be present, which ties into a manual highlighting mechanism: if the user had, during the chat, marked certain messages or parts (for instance via a future feature to star messages, akin to pinning ¹⁶), those starred items could be the only ones shown. (Pinning is not yet in the MVP, but we plan to allow marking important messages during chat for later export).

These one-click filter options mean the user doesn’t have to manually deselect lots of content. It aligns with competitor features like “export only answers” ¹⁰, but in a more interactive way where the user can see the result before exporting.

Preset Export Actions

Simple Mode streamlines common export actions into either single clicks or minimal choices:

- **Primary Export Button:** At the bottom of the panel (or top, for visibility) there is a prominent button, e.g. **“Export Conversation”**. By default, clicking this will export the *current preview content* in the default format (Markdown). Essentially, if the user just wants a quick save of the whole chat, they can open panel and hit this button immediately – one extra click to confirm format, then done. If a range or filter is active, it will export exactly what’s shown in the preview. We could label it dynamically (“Export Entire Conversation” or “Export Selected Messages” for clarity). This one-click flow addresses the primary use case of archiving the chat with no fuss.
- **Format Selection:** Next to the Export button is a small dropdown or set of icons for format selection: **Markdown (.md)**, **HTML**, **JSON**, and possibly **Plain Text**. Markdown is default since it’s both human-readable and preserves formatting (and can be easily converted to PDF or imported to other apps). HTML would produce a self-contained HTML file (with embedded CSS for the chat style). JSON gives the structured data (same format as internal representation). If the user chooses a format from the dropdown, the main button’s label or icon updates accordingly (e.g. “Export as HTML”). We don’t flood the UI with too many format buttons – one primary and a dropdown is sufficient for simplicity. *(PDF is not included initially in the UI to avoid overpromising, given complexities, but we mention it as a planned addition. Users who specifically need PDF can currently export to HTML or Markdown and convert externally.)* In future iterations, a direct PDF export could be added here once stability issues (like those noted in competitor reviews ¹²) are resolved.

- **Copy to Clipboard:** Alongside download options, a **Copy** icon is available to copy the formatted content to clipboard (most useful for Markdown or text). For instance, a user might quickly copy the conversation and paste into a document or an email. This action respects the current selection/filter as well. It's similar to ChatGPT's own "copy code" button but for the whole conversation content. (*Cloudscape design guidelines note that providing a "copy to clipboard" for AI responses is useful* ¹⁷ ; here we extend it to the whole export.)
- **One-Click Presets:** To further reduce steps, the Simple panel can present a few one-click preset actions as shortcuts (possibly at the top as buttons or in a welcome prompt in the panel):
 - **"Save Full Chat (Markdown)"** – equivalent to opening panel and immediately exporting entire convo as Markdown. This could even be exposed outside the panel: e.g. a long-press or right-click on the main Export button could trigger this directly. But within the UI, we can show it as a labeled quick action.
 - **"Save Code Snippets"** – this would automatically apply the "Code Snippets" filter and then export, possibly as a ZIP (see code workflow below).
 - **"Save This Answer"** – if the panel was opened with a particular message in mind (or if the user had one selected), this would focus that. However, since we provide a context menu for single answer, this preset might simply be for discoverability ("Did you know you can right-click an answer to export it?" as a tooltip).
 - **"Export JSON (Raw Data)"** – for developers who just want the JSON of the entire conversation for further processing, a quick button for that format is handy (this essentially bypasses the preview, as JSON doesn't need preview).

These presets act as guided shortcuts and also communicate to the user what's possible (educating them about features like code extraction). They should use simple language (avoid jargon) – e.g. "Save full chat", "Extract all code", etc., so even non-experts understand.

Transition & Feedback

When the user triggers an export (download or copy), the UI provides feedback: a brief loading spinner if needed, and a confirmation message like "Conversation exported as **ChatGPT-Example.md**". If downloading, we rely on browser's download bar; if copying, we show a small toast "Copied to clipboard". The panel remains open in case the user wants to do another export in a different format or adjust something, but it can auto-close if the user prefers (perhaps a setting for "auto close after export").

Smooth transitions are used within the panel – e.g. when toggling a filter or selecting a range, the changes reflect with a gentle animation (fading out filtered messages, etc.) to make the experience feel polished and not jarring. The design should be visually "pleasing on the eyes": using ChatGPT's modern dark theme styling, with our additional controls neatly organized. Ample spacing, rounded corners, and maybe subtle drop shadows on the overlay will make it feel like an integrated part of the ChatGPT environment, not an awkward third-party widget.

Advanced Mode Design

The **Advanced Mode** is intended for power users (developers, researchers, or meticulous note-takers) who want fine-grained control over exactly what gets exported and how. It exposes the underlying structures and offers specialized tools, while still striving for clarity.

Block-Level Selection Interface

In Advanced mode, the panel content switches to a more detailed selection interface. Key elements:

- **Content Tree / Block List:** We display the conversation broken down into a sequential list of *blocks*, similar to the current MVP but organized by message. We can visually group blocks under each message turn:
 - For example, label **User Message 1** – then indented under it list block(s) of that user message (usually just one text block unless the user message had multiple paragraphs, then each paragraph might be a block if needed). Then **Assistant Answer 1** – under it list each block of the assistant's reply (e.g. a paragraph block, a code block labeled with its filename or language, a list block, etc.). This hierarchy tree can be collapsible by message to reduce clutter.
- Each block has a **checkbox** to include/exclude it, and a text label or icon indicating its type (text, code, image, list, table, etc.). For code blocks, we include the filename if present (e.g. “☒ Code block – hello.py (version 2)”¹⁰). For text, maybe the first few words as a snippet for identification. We also show a small preview icon or ability to highlight that block in the main preview (e.g. hovering a block could scroll the chat preview to that content for context).
- **Select/Deselect Controls:** Above the list, there are helper controls: **Select All / None**, **Select Only Assistant** (to quickly tick only AI answers and untick user prompts), **Invert Selection**, etc. These match the features seen in ExportGPT¹⁰ and typical multi-select interfaces. This allows quick broad strokes, after which the user can fine-tune individual blocks.
- **Live Preview / Editor:** On the right side (if the panel is wide enough) or below (if narrow), we show a **live preview of the output** given the current selection. This can be a raw text preview in the chosen export format (e.g. if Markdown is selected, show the Markdown text; if HTML, perhaps a rendered HTML preview or the code). The user can toggle between *rendered view* and *raw text view*. For instance, if exporting to Markdown, one might want to see the rendered result (which would look like the chat bubbles as in Simple mode) or the markdown source. Power users might prefer raw text to quickly inspect for any formatting issues or to copy a portion. This preview updates in real-time as checkboxes are changed – similar to how some document export tools let you see what you'll get.
- **Edit Before Export:** Uniquely, we allow minor edits in this preview area in Advanced mode. Perhaps the user wants to tweak a heading or fix a typo before exporting; they can click into the raw text and edit it (the UI could clone it into a textbox). This is a bonus for advanced usage (like preparing a cleaned-up Markdown for publication). We will provide a clear indication that this preview has been edited (and maybe an “undo reload” if they want to revert to original content from chat).

- **Format Choice:** Just like in Simple, the format can be chosen. In Advanced, we might surface additional formats in the dropdown (e.g., plain text CSV for tables or other specialized ones) since this audience might use them. But the core remain MD/HTML/JSON.

Code Snippet Grouping & Versioning

One of the powerful features in Advanced mode is handling code snippets intelligently – a pain point for developers manually copying code piece by piece. Our design for this:

- **Code Filter & Group View:** The user can click a button “🔍 Code Snippets” (or a tab within Advanced) that switches the block list to a **code-centric view**. In this view, instead of listing conversation order, we group by **filename** (or by “Untitled code” for blocks without a specified name). For example: “**hello.py**” might have 3 versions in the conversation (perhaps the user refined it over three assistant answers). Under the **hello.py** heading, we list Version 1, Version 2, Version 3 – each corresponding to the code block from that point in the chat. Timestamps or message indices can be noted (e.g. “Version 1 – from Assistant answer #2”).
- The user can click on any two versions to **compare** them. We will then show a **diff view** side-by-side or inline: lines added in green, removed in red, similar to a Git diff. This helps the user see what changed between iterations. (If the code is large, perhaps we open a larger diff modal for better viewing.)
- There’s also an option “Select final versions only” which would automatically pick the last version of each file for export, assuming the user just wants the final code state. Conversely, “Include all versions” could package them all.
- **Exporting Code:** If a user wants to export code snippets, they have a couple of options:
 - **Embed in Document:** If they are doing a Markdown/HTML export of the whole conversation, code blocks remain in-line (as part of chat). That’s standard.
 - **Export as Files:** We provide an option to save code blocks as actual files on disk. For example, a chat that produced `hello.py` and `utils.js` can be exported so that the user gets a **ZIP file** containing `hello.py` and `utils.js` with the final code from the conversation. If multiple versions exist, we might include each version as separate files (like `hello_v1.py`, `hello_v2.py`, etc., or put them in subfolders by version) – we’ll clarify this in the UI with a checkbox “Export all versions” vs “Export latest only”. If “all versions” is chosen, we could also include a unified diff file or a changelog text summarizing differences.
 - **Copy code:** Similarly, the user could copy all code blocks to clipboard in one go (useful if they want to paste them into an editor manually).

This code-focused workflow addresses scenarios like a user asking ChatGPT to generate multiple files of a project or iterative improvements to the same snippet. It saves time by **automating what would otherwise be manual copy-paste for each block**. No existing chat exporter has this level of code awareness (as far as our research shows), giving our extension a differentiator for developers.

Additional Advanced Features

- **Custom Templates:** Advanced users might want custom formatting (for integration with blogs or reports). While not in this phase, we design with extensibility in mind – e.g., allowing a user to define

their own template for how the chat is formatted (maybe via placeholders for user/assistant). This could be a future addition in Advanced mode (not exposed now, but the architecture will not preclude it).

- **Metadata Options:** A section in Advanced mode allows toggling inclusion of metadata in exports – e.g. conversation title, date/time stamps for each message, model name, etc. Some users might want timestamps to know when each message was sent (like chat logs), others might find them noisy. We let the user decide. The JSON export of course contains full metadata (conversation ID, etc., as available). For Markdown/HTML, an option “Include timestamps” or “Include model name” can prepend that info in a small font in the bubbles or as a header.
- **Continuous Updates:** For extremely long chats or during an ongoing conversation, advanced users might want to export progressively. We might include an experimental “Auto-Export” toggle that updates the exported file as the conversation grows or at intervals. This is speculative and may not be in initial release, but mentioned as a possibility since advanced users might ask for ways to track chats over time.

Despite the complexity of Advanced mode, the UI should remain logically structured and not intimidating. By default, sections are collapsed (e.g., code grouping is hidden until clicked, metadata toggles in an accordion). We also provide a **help tooltip** or info icon next to complex features (“What is diff view?” etc.) to explain in one sentence. For instance, hovering “diff” could show “Compare two versions of a code snippet to highlight changes.”

Output Formats & File Handling

Our extension supports multiple export formats to cover common needs:

- **Markdown (.md):** The default output, as it’s versatile. The exported Markdown will be clean and well-structured: e.g., user messages prefixed with “User:” and assistant with “Assistant:” (or using blockquotes or other styling to indicate roles), or possibly using HTML blocks for the bubble styling. Lists, code, tables are in proper Markdown syntax. This ensures compatibility with platforms like Obsidian or Notion – indeed users praise Markdown exports for easy import ¹⁸. We ensure any images are included via links (if image URLs are accessible) or by embedding them as base64 (for self-contained MD, albeit that increases size).
- **HTML:** We provide an HTML export that by default uses a **chat-style theme** (CSS included) to present the conversation visually similar to the preview. This HTML can be opened in a browser and show the left-right bubble layout with colors, etc., making it ideal for sharing as a single file or printing to PDF. We also consider an alternative “plain” HTML (no bubble styling, just Q&A linear text) if users need a simpler format – but perhaps that can be an option “Simplified HTML” in advanced settings.
- **JSON:** The JSON export contains the full structured data: conversation title, timestamp, an array of message objects (with role, content, and content broken into blocks with types). Essentially the same JSON our extension uses internally. This is useful for programmatic analysis or re-importing elsewhere. We’ll document the schema in the extension docs.
- **Plain Text / CSV:** Not a primary format in UI, but if filters like “Tables only” are used, offering CSV export for those tables might be useful (e.g. user asks ChatGPT to produce a table of data, they

might want a CSV). In advanced mode, we might have a context action on a table block: “Download CSV of this table”. This is a niche but thoughtful addition for knowledge capture use case.

- **PDF (Future):** Many users ultimately want PDF for easy sharing or printing. We plan to integrate a PDF export using the HTML render behind the scenes (likely via an HTML-to-PDF library or print stylesheet) ⁷. However, given the known issues (page breaks, image encoding, large chat performance) ¹¹, we will introduce this carefully. It might start hidden under “Beta features” or advanced mode only. When implemented, it will appear as a format choice. The PDF will mirror the chat style, possibly with a header including conversation title and date.
- **Images:** The extension can already screenshot the conversation DOM for image output (as some extensions do ¹⁴). But since our focus is on text-based exports and structured data, we will not prioritize an image export button in the main UI. (Users wanting an image can still use ChatGPT’s native screenshot share, or copy the HTML into a word processor to save as image/PDF.) Nonetheless, our code will handle images *within* conversation: any inline images from the chat should be exported (either as links or embedded data URIs in HTML/PDF, and possibly downloaded separately if we zip assets).

All downloads will default to a filename derived from the conversation title (if available) or “ChatGPT_Export_[date]” plus appropriate extension. If multiple files (like code files or images) are exported, a ZIP archive is created containing them. We ensure the user is informed (“Exporting 5 files as Chat_123.zip”).

UI Layout & Navigation Considerations

Layout: The overlay panel is designed to be responsive. In a wide desktop window, it might appear as a centered modal. On narrower screens, it could dock to the side or become full-screen (like a mobile sheet). The tab switch (Simple/Advanced) is at the top, clearly visible. Within each mode, sections are logically separated with headings: e.g., in Simple mode, above the preview could be a header “Preview & Select” and then “Export Options”. In Advanced, maybe two columns (selection left, preview right). We will use icons with labels (to avoid ambiguity) for actions: e.g., a filter icon for content filters, a scissors icon for range trim, a floppy disk/download icon on the export button, etc., with hover tooltips.

Tabbed vs. New Tab: We chose a **tabbed panel** approach (versus a completely separate page for the UI) for immediacy and because users can see the result next to the original chat. This is consistent with how users interact with chat – they prefer not to navigate away during a session. However, for *very* large conversations, performance might suffer in the overlay. In those cases, an option “open in new window” (which would load a standalone HTML with the same UI) is provided. Internally, the code to render preview/export can be used in either context. This hybrid approach covers both preferences.

Contextual Menus and Quick Actions: As mentioned, right-click on an answer provides quick export. We might also integrate with ChatGPT’s UI elements; for instance, adding an **“Export” button next to the existing Copy Code button** on a code block (when you hover a code block, ChatGPT shows a copy icon – we could add a download icon there to save that code block as a file). This would be a subtle enhancement for devs: if they only need that snippet as a file, one click does it. We will do this only if it can be done cleanly without disrupting ChatGPT’s layout (likely an advanced feature toggle).

Additionally, the extension's browser toolbar icon (if clicked directly) could either open the panel (if on ChatGPT) or a help page (if outside). But since we have an in-page button, the toolbar icon is secondary.

Saving State: The extension should remember user preferences – e.g., last used format, last mode, whether they prefer including timestamps, etc. This ensures a smoother experience (power users shouldn't have to enable the same toggle every time).

Non-ChatGPT Pages: If the user tries to use the extension on a page that isn't ChatGPT (chat.openai.com), we might show a message "This extension works on ChatGPT pages. Please open ChatGPT to use." This avoids confusion if someone clicks the toolbar icon outside context.

Accessibility & Visual Design

We adhere to accessibility best practices:

- **Color & Contrast:** The panel's color scheme will automatically adjust to ChatGPT's theme (which can be light or dark). All text and icons will meet WCAG AA contrast ratios against the background. For example, in dark mode, we use light text on dark backgrounds; we'll also use distinct colors (and not rely solely on color) to differentiate user vs assistant (perhaps include labels "You/Assistant" as text for screen readers). We avoid problematic color combinations for color-blind users by using not just color but also position (left/right) and labels to identify speakers.
- **Keyboard Navigation:** The overlay can be opened via keyboard (we could assign a shortcut like Alt+E for export). Once open, the user can navigate using Tab/Shift+Tab through all controls (we'll ensure proper tabindex order). Pressing space/enter on a focused checkbox toggles it, arrow keys can be used in the block list to move selection, etc. The range selection in Simple mode will need a keyboard method as well – e.g., focus moves to the message list, and maybe pressing Enter on a message sets start, Enter on another sets end. At the very least, users can use keyboard to toggle the content filters and hit export without needing a mouse.
- **Screen Reader Labels:** We label all interactive elements with descriptive text. The floating "Export Chat" button has `aria-label="Export conversation"`. In the panel, checkboxes would have labels like "Include [Assistant answer 3, text block: 'Sure, here is the code...']". The preview is mostly read-only text which a screen reader can read as the conversation (since it's just an HTML representation). We'll make sure important status changes (like "range selected from message X to Y") are announced, possibly using ARIA live regions for notifications ("Copied to clipboard" etc.).
- **Avoiding Motion Sickness:** Any animations (like panel slide or message highlights) are kept subtle and respect user preferences (if the user has reduced motion preference, we skip animations).
- **Testing with Users:** We would ideally get feedback from both a non-technical user and a developer on this design. For instance, ensure the Simple mode truly feels simple (maybe even hide the filter options under a "More..." if we find it too much at first glance) and that Advanced mode is not confusing (the grouping and diff might require clear instructions/tooltips).

User Workflows & Scenarios

To illustrate how the design meets the initial use cases, here are typical scenarios with the user steps and UI responses:

1. Export Full Conversation (One-Click Archive)

Scenario: Jane has finished an insightful ChatGPT conversation and wants to save it for future reference. She's not technical – she just wants a document of the whole chat.

- **Step 1:** Jane clicks the **Export Chat** button at the bottom-right of the ChatGPT page.
- **Step 2:** The **Simple mode panel** opens, automatically populated with the entire conversation in chat preview form. She briefly scrolls to see it's all there in a nice bubble layout (user on right, AI on left). It looks familiar, giving her confidence.
- **Step 3:** She doesn't need any filtering or trimming, so she directly clicks the **Export Conversation** button. The format dropdown next to it shows "Markdown (MD)" as selected by default.
- **Step 4:** A file save dialog appears (since some browsers ask where to save). The suggested filename is "**ChatGPT_Jan15_2025_conversation.md**" (using the chat's title or date). She saves it.
- **Step 5:** The panel shows a small confirmation "Exported to Markdown". She closes the panel. Later, she opens the Markdown file in a viewer – it's nicely formatted with each turn labeled and the chat content intact. If she prefers, she can easily convert this to PDF or copy it elsewhere.

(This workflow took essentially two clicks after the conversation – matching the "one-click" feel. The UI made no demands on her other than confirming the action.)

2. Capture Highlights (Selective Answers)

Scenario: During a long chat, John, a researcher, received a few particularly insightful answers from ChatGPT (for example, Answer #4 and #7 in the conversation are summaries he wants to include in a report). He wants to export just those key answers, not the entire conversation.

- **Step 1:** John clicks **Export Chat** to open the panel in Simple mode. He sees the whole chat in preview.
- **Step 2:** Instead of exporting all, he uses the **Select Range / Partial** feature. He scrolls the preview to find Answer #4 (the assistant's 4th reply) which he remembers was a bulleted list of findings. He clicks the "**Select Range**" toggle, then clicks on the message bubble for Answer #4, marking it as the start. He then scrolls and clicks on Answer #7's bubble as the end.
- **Step 3:** The preview now filters to show only Answer #4 through Answer #7 (it also includes the user prompts in between by default, since a range includes everything between – but suppose he only wants the AI parts). To refine, John toggles "**Assistant Answers Only**" in the filter options. Now the preview hides his questions, showing a sequence of just the AI answers #4, #5, #6, #7. They appear as four left-side bubbles one after another. That's exactly what he needs – the highlights of the conversation.
- **Step 4:** He clicks the format dropdown and chooses **HTML**, because he wants to copy this into a report with formatting (and HTML will keep the bullet points and tables nicely).
- **Step 5:** He hits **Export as HTML**. The extension generates an HTML file (perhaps containing just those four answers, each labeled "ChatGPT Answer" and maybe a footnote that it's part of a larger convo). John saves the file and closes the panel.

- **Outcome:** He opens the HTML and sees the content styled in a clean way (each answer separated, maybe with a note “Excerpt from ChatGPT conversation”). He can copy-paste from here or attach the HTML to an email. The key point: he got just the pieces he cared about, via a visual selection that was much faster than manually copying each answer or editing a full export.

(Alternatively, John could have clicked checkboxes in Advanced mode to pick those answers, but the simple range selection with a filter was faster in this case. The design gives multiple paths to achieve the goal.)

3. Developer Code Extraction & Version Diffs

Scenario: Alice used ChatGPT to help write some code. Over a conversation, ChatGPT provided code for `module.py`, then later updated `module.py` after feedback, and also provided a `README` text. Alice wants to save the final code to run it locally, and see what changed between the first and second version of `module.py`.

- **Step 1:** Alice clicks **Export Chat**. She immediately switches to **Advanced mode** (she’s a power user and has used it before).
- **Step 2:** In Advanced mode, she clicks the **Code Snippets** view toggle. The UI now shows a list of files: e.g., **module.py (2 versions)**, **README (text)**. Under `module.py`, she sees “Version 1 (Assistant message 3)” and “Version 2 (Assistant message 5)”.
- **Step 3:** She clicks a **Compare/Diff** icon next to those versions. A diff viewer pops up in the panel (or in a modal). It highlights that in Version 2, a new function was added and some variable names changed (lines in green and red). Alice quickly reviews differences – this is helpful to recap what was modified.
- **Step 4:** She decides to export the code. She checks a box “Final versions only” (deselecting older versions). She also ticks “Include as files (zip)” option.
- **Step 5:** Now she presses **Export**. The extension compiles a ZIP file download. Inside the ZIP, there is `module.py` (containing the code from Version 2) and maybe `README.txt` (if that was recognized as a text output in a code block or just part of an answer – if it was just an assistant message, it might show up separately, but let’s assume it was a markdown code block for a README). Because she chose final only, it did not include version 1 of `module.py` separately (though we could also include a `diff.txt` inside the ZIP as an enhancement).
- **Step 6:** Alice saves the ZIP, extracts it, and now she has the code ready to run, without any ChatGPT UI cruft. The README is also there if needed.

(This scenario highlights how the extension saves developers time. Without it, Alice would have scrolled, manually copied the code from each message (hoping not to miss anything), and manually diffed them. Instead, a few clicks in the advanced panel achieved it.)

4. Quick Single-Answer Export

Scenario: Bob is chatting on ChatGPT and gets a particularly useful answer (say a step-by-step guide). He wants to save just that answer immediately to reference later, without the rest of the chat.

- **Step 1:** Without even opening the export panel, Bob **right-clicks** on the assistant’s message bubble that he wants.
- **Step 2:** In the context menu, he selects **“Export this answer (Markdown)”**. (He set Markdown as default in extension settings; or if not, the extension can default to Markdown for this quick action.)

- **Step 3:** Instantly, a `.md` file download starts, named perhaps “ChatGPT_answer_2025-08-22.md”. If the extension has the permission, it could also directly copy it to clipboard or open a save dialog depending on configuration.
- **Step 4:** Bob now has that single Q&A pair saved. If he opens the file, it might include the question for context followed by the answer formatted in Markdown (with headings, lists, etc. as originally rendered). Because this bypassed the UI, Bob didn’t get a preview, but it was a conscious quick action – and he trusts it because he’s used similar export for full chats. If something was wrong, he could still open the main panel and refine, but usually it should just work (“sensible defaults”).

(This scenario is all about speed. It leverages familiarity with right-click saving (like “Save image as...” analogously “Save answer as...”). Casual users might not discover it immediately, but we can hint at it in the UI tips. It dramatically simplifies the “just give me this answer” use case which currently one would do by copy-pasting manually.)

5. Knowledge Capture – Tables and Lists

Scenario: Carol asked ChatGPT a complex question and got a lengthy answer which includes a summary in bullet points and a results table. She only wants to keep the summary bullets and the table, ignoring the explanatory text.

- **Step 1:** Carol clicks **Export Chat** (Simple mode opens). She sees the full answer, which is long.
- **Step 2:** She toggles on the filters “**Bullet Lists**” and “**Tables**”. Immediately, the preview condenses: only the bullet list section of the answer and the table are shown, and everything else (plain paragraphs) is hidden. If these were in separate messages, it would have shown both, but in this case perhaps the assistant answer had multiple formats. The preview might show a small gap or “... content hidden ...” indicator for clarity. She reviews and indeed sees just the key points and the data table she needs.
- **Step 3:** She clicks **Export as Markdown**. The resulting Markdown file contains a neat list and a markdown table. She saves it.
- **Step 4:** Later, she opens her notes app and pastes the content – it’s already clean (no extra prose). She’s happy she didn’t need to delete unwanted text manually.

(This scenario demonstrates the content-type filtering in Simple mode. It’s especially useful for capturing structured info from a verbose chat. The UI made it a matter of toggling two buttons.)

In all these scenarios, the combination of visual preview and easy toggles/shortcuts shows how the design caters to different needs fluidly. Casual Jane didn’t need to see any complexity; Power-user Alice found advanced tools at her fingertips. The extension essentially *adapts* to the user’s level of interest in configuration.

Rationale & Justification of Design Choices

This design draws from **proven patterns** and addresses **user needs identified in research**:

- Using a **left/right chat bubble preview** is justified by common chat UX and guidelines that recommend distinguishing user vs AI messages visually ¹. This not only makes the export look

good, but also helps in the UI itself (users can easily map the preview to the actual chat). It mimics what users expect when reading a chat transcript, improving comprehension.

- Splitting **Simple vs Advanced modes** follows the principle of *progressive disclosure* in UX – showing basic options by default and revealing complexity on demand. Many competitor tools lack this separation, either being too simple (no custom options) or too complex from the start. Our approach was influenced by how some software offers “Basic” and “Expert” mode, or how Notion has simple clipping vs advanced templates – ensuring normal users aren’t scared off, while experts aren’t limited.
- The idea of **one-click presets and context actions** comes from observing user frustrations. People resorted to screenshots for quick sharing because it was the only easy method ¹⁹. By offering one-click export and right-click save, we provide a similarly quick path but with better fidelity than a screenshot. The context menu usage aligns with typical OS behavior and avoids adding extra UI elements to each message (which could clutter the interface and conflict with OpenAI’s design).
- **Filtering by content type** (code, tables, etc.) is a direct response to user desires to extract certain info. No existing extension except ExportGPT offers something like “only answers” ¹⁰, and none offer “tables only” as far as we found. But users frequently ask ChatGPT for structured outputs (lists, tables) and it makes sense to grab those without filler. This feature is inspired by web clippers and our knowledge of how users highlight key info in documents. It also leverages the structured JSON we have – we know which blocks are lists or tables, so it’s straightforward to implement and adds high value.
- The **code grouping/diff** feature is justified by developer workflows. It’s common for a single coding task to produce multiple versions of a file in one chat; grouping them prevents the user from manually collating pieces. Diffs help understand changes – something any developer using version control is familiar with. While it’s an advanced feature, we included it because it addresses a real pain (assembling complete code from chat) and sets our tool apart. We validated this idea by noting how users share code via Gists or how they might use ChatGPT Code Interpreter to analyze differences – indicating a need for tracking changes. The design follows patterns from IDEs (file tree, diff view) to feel intuitive to that audience.
- **Integration with ChatGPT’s style and resilience:** We avoid relying on exact CSS selectors of ChatGPT by using robust methods (like reading the conversation text or using data attributes). The UI elements we inject (button, overlay) are styled to look native. This is important because OpenAI occasionally changes the DOM; a brittle approach would break the extension often. By keeping our UI mostly self-contained (overlay) and using minimal hooks (the main conversation container for data), we reduce maintenance. This was a lesson from competitor issues – e.g., the “Save ChatGPT” bookmarklet broke when ChatGPT changed their HTML ²⁰. Our approach is more resilient by design.
- **Lightweight & Performance:** The overlay only renders on demand (when button clicked) and we can optimize by not re-building the preview unless needed. Also, by having the option to open a new tab for huge chats, we avoid heavy operations in the context of the single-page app that is ChatGPT. We also plan to not load any external scripts or analytics, answering some privacy concerns users

had with other extensions ¹¹. We only operate on chat.openai.com domain and do not transmit data elsewhere – alleviating privacy fears noted in reviews of other tools.

- **Originality:** While we borrow good ideas (e.g., a share link, multi-format support), we aim to deliver an original combo: no existing solution offers a visual chat preview with interactive selection *and* advanced block control in one package. The dual-mode UI and code-diff functionality are novel. Our design is not just a copy of Prompt Genius or ExportGPT – it’s a hybrid that strives to “**combine the best patterns into something more dynamic and useful,**” per the brief. For example, using a range slider concept for selecting chat segments is an innovative twist specifically tailored to chat content (much like selecting part of a video, but for text conversation).

Finally, all choices were filtered through **usability heuristics**: is it **easy to learn** (yes – basic export is obvious, advanced features are optional), **efficient to use** (one click for main tasks, batch operations for code), **error-forgiving** (preview shows exactly what you’ll get, reducing chance of exporting wrong content; undo range selection easily), and **satisfying** (the output looks good, matching expectations, which is a form of aesthetic usability). We also considered **accessibility** at each step as detailed, so no user group is left out.

With this specification, the extension will cater to a wide audience: from someone who just wants a PDF of their chat to read later, to a developer mining the chat for code, all within a simple yet powerful UI. The design is validated against what users asked for (export individual or all chats ⁷, share easily ², manage code, etc.) and what competing tools succeeded or failed in. By implementing this, we expect increased user satisfaction and a standout position among ChatGPT utilities.

Mockup Sketches (Descriptive)

Below we provide descriptive “sketches” of the UI since actual images are not included. These textual mockups illustrate the interface layout:

Simple Mode Panel (Full Conversation):

```
-----
[ Simple Export]    [⚙ Advanced]    Export Chat ✕
-----
| Chat Preview:  (Showing 8 messages)  |
|                                         |
| Me: **Question about data analysis?** |
| (right-aligned bubble)                |
|                                         |
| ChatGPT: Sure! Here are the results... |
| - Bullet point one...                  |
| - Bullet point two...                  |
| [Table: Summary of Data]              |
|                                         |
| Me: **Follow-up question.**           |
| (right-aligned)                       |
|                                         |
```

```

| ChatGPT: Certainly, here's the code:
| ```python
| (code block content)
| ```
|
| ... (continues for all messages) ...
|
-----
Filters: ☐ Users ☒ Assistants | Show only: ☐ Code ☐ Tables ☐ Lists
Range: All messages [Select Range]
Format: [Markdown ▼] [Export Conversation]
-----
Copy Markdown Export as HTML Export as JSON (etc.)

```

In this sketch: the top bar has the mode toggle and a close button. The preview shows user and assistant bubbles distinguished by alignment and maybe bold names. Filters allow toggling roles and content types. Range is currently "All" with an option to select. Bottom has format chooser and export button, and maybe secondary actions like copy or other format quick buttons.

Advanced Mode Panel (Code Group view example):

```

-----
[ Simple]          [⌕ Advanced Export]          Close X
-----
| Conversation Blocks:                                | Preview
| (Markdown):                                          |
|
|-----|
| [-] User (Q1): "How do I... ?" (1 block)            | **User:** How do
| I...                                                |
|      ☒ "How do I export data?"      | **Assistant:**
| Here is...                                          |
|
|      1. First, do X...
| [-] Assistant (A1): (3 blocks)                    | 2. Then, do Y...
|      ☒ Text: "Sure, here's how..." | ```python
|      ☒ List (3 items)             | code snippet
|      ☐ Code: script.py (version 1)         | ```
|
|      *Table: Results*
| [-] Assistant (A2): (2 blocks)                    | | Col1 | Col2 |
|      ☐ Text: "(Apology... skipped)"      | |-----|-----|
|      ☒ Code: script.py (version 2) | | Val | Val |
|
|
| [-] Assistant (A3): (1 block)                      | ... (preview
| updates as selection changes) ...
|      ☒ Table: Summary of results (5x2) |
|
|

```

[File Grouping] [v] (Switch to grouped by file view)	
script.py: [] v1 [✓] v2	Diff v1 vs v2
output.txt: [✓] v1	(shows inline differences)
[Select All] [Select None] [✓] Assistant only	

Format: [Markdown] Include: [✓] timestamps [] meta info	
[Export .ZIP] ▼	

In this sketch: The left shows the hierarchical list of content with checkboxes. User and Assistant messages can be collapsed. The code grouping toggle is on, showing files and versions (script.py v1, v2). The preview on right shows the Markdown output updating. There is an option to export as ZIP (maybe because code grouping is on, the export button adapts). The top shows we're in Advanced mode. Controls for select all/none and filtering assistant only are present.

(These textual mockups are illustrative – actual UI will be cleaner and likely use icons and better alignment – but they convey the structure and relative placement of elements.)

Conclusion

This specification provides a complete blueprint for a ChatGPT Export extension that is **user-friendly, innovative, and backed by research**. By surveying existing tools and understanding user needs, we combined their best aspects (one-click ease, multi-format flexibility, shareability) with original enhancements (visual range selection, content-type filters, code version management). The result is a cohesive UX that can **scale from the simplest use case to the most advanced**.

By implementing this design, we will enable users to effortlessly capture their ChatGPT interactions – whether it's saving an important answer, compiling a knowledge base, or extracting code – all while feeling in control and confident in the output. This empowers both everyday users and developers to get more value out of ChatGPT, bridging the gap between AI conversations and practical, shareable artifacts.

Sources: The design decisions and feature validations are supported by insights from existing solutions and user feedback, as referenced throughout this document – for example, one-click sharing via ShareGPT ², multi-format export demands including PDF ⁷, the success of Markdown exports in tools like ExportGPT ¹⁸, and the need for highlighting key info akin to pinning ¹⁶. These references ensure our approach is grounded in real user expectations and proven patterns, not just theoretical. With this well-researched foundation, the extension is poised to deliver a superior experience in exporting ChatGPT conversations.

¹ ¹⁵ ¹⁷ Generative AI chat - Cloudscape Design System

<https://cloudscape.design/patterns/genai/generative-AI-chat/>

² ³ ⁴ ⁵ ⁶ ChatGPTの履歴をエクスポートして共有するための4つのアプリと拡張機能

<https://www.tunesbro.jp/blog/4-apps-and-extensions-to-export-and-share-your-chatgpt-history.html>

7 8 20 **Request to Export Individual ChatGPT Conversations : r/ChatGPT**

https://www.reddit.com/r/ChatGPT/comments/1kh9zy6/request_to_export_individual_chatgpt_conversations/

9 10 11 12 18 **ExportGPT - Export ChatGPT Conversations Easily**

<https://chrome-stats.com/d/jamcijfplmgbngnppdhmbboggjebgfimn?hl=en>

13 **Export your workspace data | Slack**

<https://slack.com/help/articles/201658943-Export-your-workspace-data>

14 **chatGPT Screenshot - Full Chat Capture Tool - Chrome-Stats**

<https://chrome-stats.com/d/nobfenidifcdghhkifkcnlhmnlnfmhek>

16 **GitHub - saeedezzati/superpower-chatgpt: ChatGPT with superpowers! Search chat history, create folders, export all chats, pin messages, access thousands of community prompts, incognito mode, language and tone selection, and many more features**

<https://github.com/saeedezzati/superpower-chatgpt>

19 **Improve Your ChatGPT with Web Highlights Highlighting Power**

<https://web-highlights.com/blog/how-web-highlights-can-enhance-your-chatgpt-experience-the-power-of-text-highlighting/>